



Master Specification: The "Cognitive 4D Memory Layer"

Version: 1.0 (Enterprise-Grade Agentic Second Brain via MCP) **Architecture Style:** Decoupled Modular Monorepo, Bi-Temporal Property Graph, Hybrid Vector Search **Core Principle:** *Memory is not the thinker. It is the highly structured, self-aware observer.*



1. Executive Summary & Core Philosophy

ระบบนี้คือ "เซิร์ฟเวอร์หน่วยความจำส่วนกลาง" ที่ออกแบบมาเพื่อรองรับ AI Agent ทุกประเภท (LangGraph, CrewAI, Autogen, Claude Desktop) ผ่านมาตรฐาน MCP (Model Context Protocol)

ปรัชญาหลักของสถาปัตยกรรม (Core Philosophies):

- Decoupled Architecture (แยกสมองแยกความจำ):** ระบบนี้จะไม่ Agent ทำงานอยู่ภายใน ไม่มีสิทธิ์ตัดสินใจแทนผู้ใช้ ทำหน้าที่เพียงรับข้อมูลดิบ, สกัดความรู้, จัดเก็บ, และเสิร์ฟข้อมูลที่แม่นยำที่สุดกลับไป
- Deterministic Over Probabilistic (ความแน่นอนต้องมาก่อน):** การไล่หาความสัมพันธ์ (Traversal) และโครงสร้างเอกสาร (Hierarchy) จะต้องใช้ **อัลกอริทึมและโค้ด** (เช่น Cypher Query, AST) เท่านั้น ห้ามให้ LLM เดาโครงสร้างเด็ดขาด LLM จะถูกใช้เฉพาะงานทำความเข้าใจความหมาย (Semantic) เท่านั้น
- 1-Size-Fits-All Storage:** ข้อมูลทุกสกุล (Text, PDF, Images, Codebase) จะถูกหลอมรวมลงใน **Vector-Enabled Bi-Temporal Property Graph** (กราฟที่มี Vector และประทับตราเวลา) ก้อนเดียว
- Self-Aware Data (ข้อมูลที่ตระหนักรู้):** ข้อมูลทุกชิ้นจะถูกวิเคราะห์มิติเชิงลึก (Cognitive Dimensions) ตั้งแต่ตอนนำเข้า เพื่อให้ Agent ภายนอกรู้ว่าข้อมูลไหนเชื่อถือได้ ข้อมูลไหนเป็นความลับ ข้อมูลไหนเป็นแค่อารมณ์ชั่ววูบ



2. Modular Package Architecture (โครงสร้างโค้ด)

โปรเจกต์จะถูกสร้างเป็น **Monorepo** (เช่น ใช้ Turborepo) แบ่งย่อยเป็น 4 Packages ที่ทำงานเป็นอิสระต่อกัน เพื่อความยืดหยุ่นในการสับเปลี่ยนเทคโนโลยีในอนาคต



2.1 @memory-layer/parser (The Deterministic Extractor)

ทำหน้าที่รับไฟล์ดิบและแปลงเป็นโครงสร้าง Text พื้นฐาน โดยปราศจากการเดาของ AI

- Input:** Raw Buffers (PDF, DOCX, Source Code, Images)
- Output:** `Array<ChunkObject>`
- Core Logic:**
- Text/Office:** ใช้ `Docling` หรือ `Unstructured` สกัดย่อหน้าและตาราง

- **Source Code:** ใช้ `tree-sitter` ข่าแหละ Abstract Syntax Tree (AST) เพื่อหา Imports, Class, Functions
- **Images/Scans:** (ข้อยกเว้นที่ใช้ AI) โยนเข้า `Gemini 1.5 Flash` ทำ Vision OCR พร้อมรักษา Layout

🧠 2.2 @memory-layer/cognitive-extractor (The AI Typist)

ทำหน้าที่รับ Text ธรรมดา มาสกัดเป็น Knowledge Graph Schema พร้อมประเมินมิติเชิงรับรู้

- **Input:** `Array<ChunkObject>`
- **Output:** `GraphDocument` (JSON Schema ที่ผ่านการ Validate ด้วย TypeBox/Zod)
- **Core Logic:**
- ส่ง Text เข้า `Cohere embed-multilingual-v3.0` เพื่อสร้าง Vector 1024 มิติ (รองรับไทย, จีน, ญี่ปุ่น, อังกฤษ)
- ส่ง Text เข้า `LLM` (Information Extraction Model — เลือกได้ตามความเหมาะสม) เพื่อทำ Information Extraction (สกัดโหนดและเส้นความสัมพันธ์)

🗄️ 2.3 @memory-layer/storage-adapter (The Graph Engine)

ทำหน้าที่รับ JSON Schema มาแปลงเป็น Cypher Query และยิงเข้า Database

- **Input:** `GraphDocument` (จาก Extractor)
- **Output:** Success/Fail (Void), `RetrievedContext` (ขาออก)
- **Core Logic:**
- จัดการ Entity Resolution: เช็ค Vector ความคล้ายคลึงก่อนสร้างโหนดใหม่ เพื่อจับยูนรวม (Merge) โหนดที่ชื่อต่างกันแต่ความหมายเหมือนกัน (เช่น "กทม." กับ "Bangkok")
- ทำ Transaction Management สร้างเส้นกึ่งก้านสาขาของ `Neo4j` อย่างปลอดภัย

🔌 2.4 @memory-layer/mcp-server (The Agent Gateway)

ทำหน้าที่รับ API Server (ผ่าน `ElysiaJS`) เพื่อเปิดรับ Request จาก Agents

- **Input:** JSON-RPC Request (MCP Standard)
- **Output:** Context Data สำหรับ Agent
- **Core Logic:** จัดการระบบ Queue ด้วย `BullMQ` (กันเซิร์ฟเวอร์รวมนานาเข้าหมีนเอกสาร) และ Map ฟังก์ชันจาก Storage Adapter ให้กลายเป็น MCP Tools

📁 3. Master Data Schema (โครงสร้างฐานข้อมูลแบบ 4D)

ฐานข้อมูลหลัก (Neo4j) จะมีโครงสร้างที่ตายตัวดังนี้

🟢 Nodes (โหนดหลัก 4 ประเภท)

1. **(:Episode):** บันทึก "เหตุการณ์" [Properties: id, timestamp, source (เช่น 'chat', 'upload'), summary]

2. (**:Document**): บันทึกข้อมูลดั้งต้น [Properties: id, title, type, language, author]
3. (**:Chunk**): บันทึกก้อนเนื้อหา [Properties: id, text, sequence_order, embedding (Vector 1024d)]
4. (**:Entity**): นามเฉพาะต่างๆ [Properties: id, name, type (Person, Org, Feature, Function), description]

Edges & The 6 Cognitive Dimensions (เส้นความสัมพันธ์ + 6 มิติ)

เส้นความสัมพันธ์แบ่งเป็น 3 กลุ่ม:

- **Structural**: [:HAS_CHUNK], [:NEXT_CHUNK], [:CONTAINS_CODE]
- **Semantic**: [:MENTIONS], [:RELATED_TO], [:ACTS_AS]
- **Temporal**: [:EXTRACTED_DURING]

 **กฎเหล็กของระบบ:** ทุกๆ เส้นความสัมพันธ์สาย Semantic ต้องมี **Properties 8 ตัว** ต่อไปนี้เสมอ:

1. **valid_from** (ISO String): เวลาที่ความสัมพันธ์นี้เกิดผล
2. **valid_to** (ISO String | Null): เวลาที่ความสัมพันธ์นี้สิ้นสุด (ถ้ายังเป็นจริงอยู่ให้เป็น Null)
3. **confidence_score** (0.0 - 1.0): ความมั่นใจในการสกัดข้อมูลของ LLM หรือความน่าเชื่อถือของ Source
4. **intent_category** (Enum): ประเภทของเนื้อหา [FACT, POLICY, DECISION, OPINION, TASK]
5. **criticality_score** (0.0 - 1.0): ระดับความสำคัญ (คอขาดบาดตายแค่ไหน)
6. **sentiment** (Enum): อารมณ์ของข้อมูล [POSITIVE, NEGATIVE, NEUTRAL]
7. **clearance_level** (1 - 4): **Metadata เท่านั้น** — ระบบหน่วยความจำนี้เป็นแค่ฐานข้อมูล ไม่มีกลไก Enforce สิทธิ์ คำนี้คือ "คำแนะนำ" ที่ Agent หรือระบบภายนอกจะนำไปใช้กรองข้อมูลเองตามนโยบายของตัวเอง
8. **expires_at** (ISO String | Null): วันหมดอายุของข้อเท็จจริงนี้

4. System Output Schema (Prompt บังคับ LLM ตอนนำเข้าข้อมูล)

ใน Pipeline ของ @memory-layer/cognitive-extractor มี 2 ขั้นตอนหลัก:

1. **Embedding**: ส่ง Text ของแต่ละ Chunk เข้า **Cohere embed-multilingual-v3** (1024 มิติ) เพื่อสร้าง Vector ก่อน
2. **Extraction**: ส่ง Text เดิมเข้า LLM เพื่อสกัด JSON Schema ด้านล่าง

นี่คือ JSON Schema ที่เราจะบังคับให้ LLM คายออกมา (ใช้ร่วมกับ Zod):

```
JSON
{
```

```

"entities": [
  {
    "id": "entity_01",
    "name": "ทริสสิน",
    "type": "Person",
    "description": "พนักงานฝ่าย IT"
  }
],
"relationships": [
  {
    "source_id": "entity_01",
    "target_id": "entity_02",
    "type": "BOARD_MEMBER",
    "valid_from": "2024-01-01T00:00:00Z",
    "valid_to": null,
    "confidence_score": 0.8,
    "intent_category": "FACT",
    "criticality_score": 0.6,
    "sentiment": "NEUTRAL",
    "clearance_level": 2,
    "expires_at": null,
    "justification": "ดึงข้อมูลจากโครงสร้างองค์กรล่าสุด ระบุชัดเจนว่าเป็นบอร์ดบริหาร"
  }
]
}

```

5. MCP Tools Specification (เครื่องมือสำหรับ Agent)

API ที่เปิดให้ Agent ภายนอกเรียกใช้งาน แบ่งเป็น 3 กลุ่ม:

5.1 Write Tools (Data Ingestion)

- **ingest_document(file_buffer, metadata)**: อัปโหลดไฟล์เข้าสู่ E-T-L Pipeline (ทำงานแบบ Async ผ่าน BullMQ)
- **memorize_fact(subject, predicate, object, context)**: ฝังความจำสั้นๆ จากแชท (เช่น "ฉันชอบกินกาแฟ") ระบบจะทำ Bi-temporal Update อัตโนมัติ (อัปเดตเส้นเก่า สร้างเส้นใหม่)
- **log_episode(summary, timestamp)**: สร้างโน้ต (:Episode) เพื่อบันทึกประวัติเหตุการณ์

5.2 Discovery Tools (Explore Before Search)

- **justify_intent(user_query)**: ส่งคำถามให้ Dispatcher วิเคราะห์ว่า Agent ควรเรียกใช้ Tool อ่านข้อมูลตัวไหน
- **discover_nodes(keyword)**: ค้นหา Entity แบบหยาบ คั่นคำ Menu ให้อัปเดตให้ Agent ยืนยันตัวตน (ป้องกัน Hallucination จากการสะกดผิด)



5.3 Read Tools (Data Retrieval)

- `semantic_search(query, lang_filter, min_clearance_level, active_only):`
- กลไก: แปลง Query เป็น Vector -> ค้นหา (:Chunk) -> Filter กรองความลับ (Clearance) และเอาเฉพาะข้อมูลปัจจุบัน (`valid_to = null`)
- `get_document_tree(document_id):`
- กลไก: ใช้ Cypher ไต่กิ่ง [:NEXT_CHUNK] คืบข้อความทั้งฉบับโดยไม่ขาดตกบกพร่อง
- `get_entity_timeline(entity_id, relationship_type):`
- กลไก: ดึงความสัมพันธ์รอบๆ Entity จัดเรียงตาม `valid_from` เพื่อสร้าง Timeline ประวัติศาสตร์
- `get_code_dependencies(function_node_id, direction):`
- กลไก: ไต่กราฟ AST ([:CALLS], [:IMPORTS]) หาผลกระทบเมื่อมีการแก้ไขโค้ด



6. Enterprise Master Scenarios (ทดสอบการใช้งาน)

นี่คือ 6 สถานการณ์ที่ระบบนี้ออกแบบมาเพื่อเอาชนะข้อจำกัดของ RAG ธรรมดา:

1. "TOR นี้ มีกี่ข้อ มีจุดไหนที่ Product ของเราไม่มีบ้าง"
 - การทำงาน: Agent เรียก `get_document_tree` ได้โครงสร้างข้อความแบบ 100% (Deterministic) แล้วเรียก `semantic_search` หาฟีเจอร์ Product จากนั้นนำมาเทียบกับใน Context Window
2. "ขอสรุปรายการ Feature Product พร้อม Roadmap"
 - การทำงาน: Agent เรียก `semantic_search` แบบเปิดโหมด `active_only=true` (เอาเฉพาะแผนปัจจุบัน) และฟิลเตอร์ `intent_category="DECISION"` ทั้งพวกที่เป็นความคิดเห็น (OPINION) ไปจนหมด
3. "นายทวิสิน เข้าไปเป็นกรรมการที่ไหนเมื่อไหร่บ้าง"
 - การทำงาน: Agent เรียก `discover_nodes` ได้ ID ของนายทวิสิน -> เรียก `get_entity_timeline` ได้เส้นความสัมพันธ์พร้อมตราเวลา -> Agent สังเคราะห์ประโยคตอบ พร้อมอ้างอิง `confidence_score` ให้ผู้ใช้ทราบความน่าเชื่อถือ
4. "วันก่อนผมสั่งงานอะไรไปบ้าง"
 - การทำงาน: Agent ค้นหาโหนด (:Episode) ที่ผ่านไปตามเวลา แล้วไต่เส้นกลับไปหาข้อความแชทที่มี Property `intent_category="TASK"`
5. "Scenario สเกลโค้ดระดับชาติ: ถ้ารู้ Auth API ตัวนี้จะฟังไปถึงไหน"
 - การทำงาน: ข้ามกระบวนการ AI ทั้งหมด Agent เรียก `get_code_dependencies` วิ่งไต่กราฟที่เกิดจาก `tree-sitter` ได้ผลลัพธ์เป็นโครงข่าย Dependency ระดับฟังก์ชันแบบเป๊ะๆ 100%
6. "ค้นหานโยบายภาษาไทยที่สอดคล้องกับ Policy_JP (10,000+ Docs)"

- **การทำงาน:** Agent โยนรหัส Policy_JP เข้ามา ระบบสกัด Vector ด้วย Cohere แล้วยิงค้นหาเฉพาะกลุ่ม `language="TH"` โดยฟิลเตอร์ `clearance_level` ให้ตรงกับสิทธิ์ของผู้ถาม (Security Guardrails ทำงานระดับ Database)
-

7. บทสรุป และ Roadmap การพัฒนา

สถาปัตยกรรม **Cognitive 4D Memory Layer** นี้ไม่ใช่เพียงแค่ฐานข้อมูล แต่เป็น **"ระบบประสาทส่วนกลาง"** ของ AI Enterprise

สิ่งที่ทีมพัฒนาควรทำเป็นลำดับต่อไป (Sprint 1):

1. ตั้งโปรเจกต์ Monorepo (Bun + TypeScript)
2. เริ่มเขียน `@memory-layer/cognitive-extractor` โฟกัสที่การทำ Zod Schema และยิง Groq API เพื่อสกัด Text ออกมาให้ได้ JSON ตามโครงสร้าง
3. นำ JSON ไปลองยัดลง Neo4j Desktop ภายในเครื่อง (Local) ผ่าน `@memory-layer/storage-adapter` เพื่อทดสอบประสิทธิภาพของ Cypher Query

เนื่องจากสถาปัตยกรรม **"Cognitive 4D Memory Layer"** ที่เราออกแบบกันไว้นั้นเป็นระดับ **Custom Enterprise Architecture** จึงยังไม่มี Repository เดียวๆ ตัวไหนในโลกที่ทำสำเร็จรูปแบบ 100% ตามสเปคนี้ครับ (ถ้าคุณทำเสร็จ มันจะเป็น Open-Source ที่ล้ำมากๆ)

แต่เพื่อให้คุณสามารถเริ่มเขียนโค้ดได้ทันทีโดยไม่ต้องคลำทางเอง นี่คือ **แหล่งรวม Reference Code Repositories** ที่ผมคัดมาให้ตรงกับโครงสร้างแต่ละ Package ของเรามากที่สุดครับ:

1. ฝั่ง MCP Server & API Framework

ดูเพื่อศึกษาการเปิดพอร์ตให้ Agent เข้ามาคุย และการเชื่อมต่อระบบด้วย TypeScript ความเร็วสูง

- [modelcontextprotocol/servers](#)
- **โฟกัส:** เข้าไปดูโฟลเดอร์ `src/` ของเซิร์ฟเวอร์ตัวอย่าง (เช่น `sqlite` หรือ `postgres`) เพื่อดูวิธีการใช้ `@modelcontextprotocol/sdk` ว่าเขาประกาศ `ListTools` และ `CallTool` กันยังไง
- [elysiajs/elysia](#)
- **โฟกัส:** ศึกษาการทำ Type Validation ด้วย `t` (TypeBox) เพื่อรับประกันว่า Payload ที่ Agent ส่งมา (หรือ LLM สกัดมา) ตรงตาม Schema ของเรา 100%

2. ฝั่ง Knowledge Graph Extraction (สกัดข้อมูลเป็น Graph)

ดูเพื่อศึกษา **"Prompt Engineering"** และการจัดฟอร์แมต JSON บังคับ LLM

- [safishamsi/graphify](#)
- **โฟกัส:** แยกโค้ดในไฟล์ `prompts.py` และ `graphify.py` เพื่อขโมยไอเดียการเขียน System Prompt ให้ LLM สกัด Entity และ Relationship ออกมาเป็น JSON (ซึ่งเราจะเอามาอัปเดตเพิ่มมิติ Confidence และ Temporal เข้าไป)
- [HKUDS/LightRAG](#)
- **โฟกัส:** ดูเทคนิคการแยก Chunking และการสร้าง "Entity Resolution" (การจับยุบรวมโหนดที่ชื่อคล้ายกันด้วย Vector)

3. ฝั่ง Bi-Temporal & Graph Database (การจัดการเวลาและฐานข้อมูล)

ดูเพื่อศึกษาการเขียน **Cypher Query (Neo4j)** ที่มีมิติเรื่องเวลาเข้ามาเกี่ยวข้อง

- [getzep/graphiti](#)
- **โฟกัส:** นี่คือสมบัติล้ำค่า! เข้าไปดูในโฟลเดอร์ `graphiti/core/` เพื่อดูการออกแบบ Schema เรื่อง `valid_from`, `valid_to` และ `Episode` โหนด ว่าเขาอัปเดตสถานะกราฟกันยังไงโดยไม่ลบข้อมูลเก่าทิ้ง

- [neo4j-labs/agent-memory](https://neo4j-labs.github.io/agent-memory/)

- **โฟกัส:** ดูวิธีการเขียน Cypher Query (ผ่าน Python/TS Driver) เพื่อ Insert ข้อมูลแบบ MERGE และการทำ Vector Search ควบคู่กับการได้ Graph แบบ Native ใน Neo4j

⚙️ 4. ผัง Document & Code Parsers (แกะไฟล์โดยไม่ใช้ LLM)

ดูเพื่อนำมาใช้ใน Layer แรกสุด (การทำความสะอาดข้อมูลก่อนส่งให้ AI)

- [DS4SD/docling](https://github.com/DS4SD/docling)

- **โฟกัส:** ไลบรารีจาก IBM ที่แกะ PDF/Word/PPT ออกมาเป็น Markdown ได้เนียนที่สุดในตอนนี้ (รองรับตารางได้ดีมาก) เอาไว้ทำ Data Ingestion

- [tree-sitter/tree-sitter](https://github.com/tree-sitter/tree-sitter)

- **โฟกัส:** เครื่องมือมาตรฐานโลกสำหรับการแกะ Codebase หาไฟล์ที่เกี่ยวข้อง (Bindings สำหรับ Node.js/Bun มีพร้อมใช้งาน) นำมาใช้สร้าง AST Graph โดยไม่ต้องแปลง Token AI

💡 คำแนะนำในการเริ่มตั้งไข่โปรเจกต์ (Starting Template)

ถ้าคุณอยากขึ้นโครงโปรเจกต์ไวๆ ผมแนะนำให้ Clone Template แนวๆ **Bun + Elysia** มาก่อน แล้วติดตั้ง Dependencies ชุดแรกเพื่อทดสอบระบบตามนี้ครับ:

```
Shell
# 1. สร้างโปรเจกต์
bun create elysia memory-layer-mcp
cd memory-layer-mcp

# 2. ติดตั้ง SDK หลักที่ต้องใช้ทำ PoC
bun add @modelcontextprotocol/sdk neo4j-driver groq-sdk
```

จากนั้นลองสร้าง Tool ง่ายๆ ผ่าน MCP ขึ้นมา 1 ตัว เช่น **ping_memory** เพื่อทดสอบว่าฝั่ง Agent (เช่น ใช้ Claude Desktop) สามารถมองเห็น Tool ของคุณและยิง Request เข้ามาถึง Server Elysia ได้สำเร็จ

ถ้าเส้นทางเชื่อมต่อ MCP ทะลุหากันแล้ว การยัด Logic ของ Neo4j และ LLM เข้าไปตรงกลางก็จะเป็นเรื่องสนุกและเป็นระบบระเบียบมากครับ!